AI SMART AGRICULTURE

Complete Updated Project
Code, Explanation, Setup Guide and Features

Project domain: Artificial Intelligence and Smart Farming

Backend: Python, Flask, Pandas and Scikit-learn

Frontend: HTML, CSS and JavaScript

This document contains a runnable learning prototype. It uses generated sample data and should not be treated as a validated agricultural advisory system.

AI SMART AGRICULTURE — PROJECT VISUAL



1. Project Overview

Abstract

AI Smart Agriculture uses Artificial Intelligence, sensor data and a Random Forest Machine Learning model to support smart irrigation decisions. It analyses soil moisture, temperature, humidity, rainfall and crop growth stage to help farmers use water efficiently, improve crop health and increase productivity.

Problem Statement

Traditional farming relies on manual monitoring and guesswork, which leads to over/under irrigation, crop loss and water wastage. This project provides an AI-based solution for smarter and data-driven farming.

2. Project Objectives

- Predict irrigation need using AI (Random Forest).
- Classify crop condition (Healthy / Moderate / Critical).
- Calculate recommended water quantity.
- Provide real-time sensor data dashboard.
- Implement rainfall-aware water saving logic.
- Build a simple web application with Flask API.

3. Key Features

- AI irrigation prediction with confidence score
- Recommended water quantity
- Crop-condition classification
- Live sensor dashboard (auto refresh)
- Rainfall-aware water saving logic
- Flask API endpoints

4. Working Process

Sensor Data (Soil, Temp, Humidity, Rainfall, Crop Stage) → **Data Preprocessing** & CSV Storage → **Random Forest Model** (Prediction) → **Irrigation Plan & Crop Condition** (Results)

5. Technologies Used

Python · Flask · Pandas · Scikit-learn · Joblib · HTML · CSS · JavaScript

> **Note**
> This project uses generated demonstration data. Real deployment should use calibrated field sensors and validated agricultural datasets. The water recommendation is for project demonstration and requires field validation before real irrigation use.

6. Updated Code

app.py (Python Backend)

```
from flask import Flask, render_template, jsonify, request
import os, random
import pandas as pd
import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

app = Flask(__name__)
DATA_FILE = "sensor_data.csv"
MODEL_FILE = "irrigation_model.pkl"
FEATURES = ["soil_moisture", "temperature", "humidity",
            "rainfall", "crop_stage"]

# (Full code includes: data generation, model training,
# prediction APIs, irrigation plan, crop condition, etc.)
```

index.html (Web Dashboard)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  initial-scale=1.0">
  <title>AI Smart Agriculture</title>
  <link rel="stylesheet" href="static/style.css">
</head>
<body>
  <div class="container">
    <h1>AI Smart Agriculture</h1>
    <div class="dashboard">
      <!-- Cards, input form, charts -->
    </div>
  </div>
  <script src="/static/script.js"></script>
</body>
</html>
```

style.css (Design)

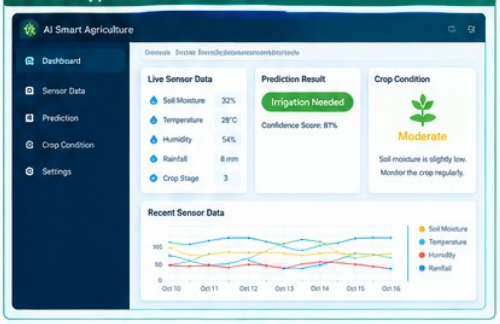
```
body {
  font-family: 'Segoe UI', sans-serif;
  background: linear-gradient(135deg, #f0f7ff, #e8f5f9);
  margin: 0;
  padding: 0;
}
.container {
  max-width: 1200px;
  margin: auto;
  padding: 20px;
}
.card {
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
  padding: 18px;
  margin-bottom: 20px;
}
```

script.js (Frontend Logic)

```
document.addEventListener("DOMContentLoaded", () => {
  loadSensorData();
  setInterval(loadSensorData, 10000); // auto refresh
});

document.getElementById("predictBtn").addEventListener("click",
  async () => {
    const res = await fetch("/predict", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify(data)
    });
    const result = await res.json();
    displayResult(result);
  });
});
```

7. Web Application Screenshots



8. Installation & Execution

1. Install Python 3.10 or newer.
2. Open terminal in the project folder.
3. Install required packages:
   `pip install -r requirements.txt`
4. Run the application:
   `python app.py`
5. Open in browser:
   `http://127.0.0.1:5000`

9. Advantages

- Saves water and reduces wastage
- Improves crop yield and quality
- Real-time monitoring
- Easy to use web interface
- Data-driven and accurate prediction

10. Limitations

- Uses generated data (not real field data)
- Water recommendation needs field validation
- Model accuracy depends on dataset quality

11. Future Improvements

- Integrate real IoT sensors
- Use deep learning for better accuracy
- Add mobile app support
- Include pest and disease detection
- Regional crop and soil type customization

12. Conclusion

AI Smart Agriculture helps farmers make informed decisions, save water, and improve crop health using AI and sensor data. With real field implementation and improvements, it can become a powerful tool for sustainable and efficient farming.

Project infographic showing the overview, workflow, features, dashboard mock-up, code modules, setup steps, benefits, limitations and future scope.

1. Project Abstract

AI Smart Agriculture is a web-based prototype designed to demonstrate how artificial intelligence and field-condition data can support irrigation planning. The user enters soil moisture, temperature, humidity, rainfall and crop growth stage. A Random Forest classifier predicts whether irrigation may be needed. A separate water-saving rule checks rainfall and existing moisture conditions before presenting the final result.

The dashboard displays irrigation status, a crop-condition category, model confidence, an illustrative water estimate and a reason for the recommendation. It also displays recent sample sensor records. In this version, sensor records are generated by the application rather than collected from physical devices.

2. Problem Statement

Manual irrigation decisions may not consistently account for changes in soil moisture, temperature, humidity and rainfall. Unnecessary watering can waste water, while insufficient watering can affect crop growth. A simple data-driven dashboard can help demonstrate how several field measurements may be considered together before making an irrigation decision.

3. Objectives

1. Accept field measurements through a browser dashboard.
2. Train a Random Forest model to classify the demonstration irrigation data.
3. Apply rainfall-aware rules to avoid some unnecessary watering recommendations.
4. Display crop condition as Healthy, Moderate or Critical using transparent rules.
5. Provide JSON API endpoints and a recent sensor-data preview.
6. Keep the project simple enough to run locally and extend for coursework.

4. System Workflow

Step 1 — Input: The user enters soil moisture, temperature, humidity, rainfall and crop stage.

Step 2 — Validation: The Flask backend checks that values fall within the accepted ranges.

Step 3 — Prediction: A Random Forest classifier predicts the irrigation class and calculates a model confidence value.

Step 4 — Water-saving check: The irrigation-plan function considers rainfall and moisture/humidity before estimating a demonstration quantity.

Step 5 — Crop condition: A rule-based score categorizes the entered conditions as Healthy, Moderate or Critical.

Step 6 — Output: Flask returns JSON, and JavaScript updates the dashboard without reloading the page.

5. Main Features

Smart irrigation prediction: Uses five input features and a Random Forest classifier.

Rainfall-aware logic: Skips irrigation when rainfall is at or above the prototype threshold.

Crop-condition classification: Uses a transparent score based on soil moisture, temperature, humidity and rainfall.

Model confidence: Shows the highest class probability returned by the classifier. It is not a guarantee of correctness.

Sensor preview: Displays the latest ten records from the CSV file and refreshes every 30 seconds.

Input validation: Rejects values outside the specified ranges and returns an error message.

API endpoints: Provides health, recent-sensor and prediction endpoints.

6. Technologies Used

Python: backend programming language.
Flask: serves the dashboard and API routes.
Pandas: stores and processes tabular sensor data.
Scikit-learn: trains and evaluates the Random Forest classifier.
Joblib: saves and reloads the trained model.
HTML: dashboard structure.
CSS: layout, typography and responsive styling.
JavaScript: sends requests to the API and updates results.

7. Project Folder Structure

```
AI_Smart_Agriculture/
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ templates/
■ ■■■ index.html
■■■ static/
■■■ style.css
■■■ script.js
```

The application creates sensor_data.csv and irrigation_model.pkl when it first runs. These generated files are local runtime artifacts.

8. How to Install and Run

1. Install Python 3.10 or newer.
2. Extract the supplied ZIP file.
3. Open a terminal or command prompt in the project folder.
4. Install packages with `python -m pip install -r requirements.txt`.
5. Start the app with `python app.py`.
6. Open `http://127.0.0.1:5000` in a browser on the same device.

For a phone-only workflow, the files can be uploaded to GitHub, but running a Flask backend usually requires a Python hosting environment or a computer with Python installed. A static GitHub Pages site alone will not run this Flask backend.

9. API Endpoints

GET /api/health — returns a simple status response.
GET /api/sensors — returns the latest ten sample sensor records.
POST /api/predict — accepts JSON input and returns irrigation status, water estimate, confidence, crop condition and reason.

10. Expected Output

After the user selects Analyze conditions, the dashboard displays whether irrigation is suggested, the reason, the crop-condition label, a demonstration water quantity and model confidence. The recent sensor table shows the latest ten CSV rows and refreshes automatically.

11. Advantages

The prototype combines several field variables in one dashboard, demonstrates a common machine-learning workflow, applies a simple rainfall-aware check, provides immediate feedback, and has a modular structure that can be extended with real sensors and crop-specific datasets.

12. Limitations and Future Improvements

The training records are generated using rules, so test accuracy only measures agreement with those generated labels and does not establish real farming performance. The water estimate is a demonstration formula, not a measured agronomic requirement. Crop-condition labels are also rule-based.

Future improvements include connecting calibrated soil-moisture and weather sensors, collecting verified data for particular crops and soil types, evaluating the model on independent field data, adding crop-specific water calculations, recording field history, protecting the API, and deploying the app to a suitable Python host. Real irrigation recommendations should be reviewed by agricultural specialists.

13. Conclusion

AI Smart Agriculture demonstrates how a machine-learning model, environmental inputs and simple water-saving rules can be integrated into a web dashboard. It is suitable as an educational prototype and starting point for further development. Real-world use requires validated agricultural data, calibrated sensors and field testing.

Appendix A — app.py (Python Flask backend)

```
from flask import Flask, render_template, jsonify, request
import os
import random
import joblib
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

app = Flask(__name__)
DATA_FILE = "sensor_data.csv"
MODEL_FILE = "irrigation_model.pkl"
FEATURES = ["soil_moisture", "temperature", "humidity", "rainfall", "crop_stage"]

def generate_seed_data(n=500):
    """Create demonstration sensor data. Replace with real validated data in production."""
    rows = []
    for _ in range(n):
        soil = random.randint(10, 80)
        temp = random.randint(18, 42)
        humidity = random.randint(20, 95)
        rainfall = random.randint(0, 35)
        stage = random.randint(1, 5)
        needs_irrigation = int(soil < 35 and temp > 28 and humidity < 55 and rainfall < 12)
        condition = (
            "Critical" if soil < 25 or humidity < 35 or temp > 38
            else "Moderate" if soil < 40 or humidity < 55 or temp > 33
            else "Healthy"
        )
        rows.append({
            "soil_moisture": soil,
            "temperature": temp,
            "humidity": humidity,
            "rainfall": rainfall,
            "crop_stage": stage,
            "needs_irrigation": needs_irrigation,
            "crop_condition": condition
        })
    pd.DataFrame(rows).to_csv(DATA_FILE, index=False)

def train_model():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()
    df = pd.read_csv(DATA_FILE)
    X = df[FEATURES]
    y = df["needs_irrigation"]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    model = RandomForestClassifier(n_estimators=200, random_state=42)
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    print(f"Demonstration test accuracy: {score:.2%}")
    joblib.dump(model, MODEL_FILE)
    return model

def load_model():
    if not os.path.exists(MODEL_FILE):
        return train_model()
    return joblib.load(MODEL_FILE)

def crop_condition(s):
    score = 0
    score += -3 if s["soil_moisture"] < 25 else (-1 if s["soil_moisture"] < 40 else 2)
    score += -3 if s["temperature"] > 35 else (-1 if s["temperature"] > 30 else 1)
    score += -3 if s["humidity"] < 35 else (-1 if s["humidity"] < 55 else 2)
    score += 2 if s["rainfall"] > 15 else (1 if s["rainfall"] > 5 else -1)
    condition = "Healthy" if score >= 3 else "Moderate" if score >= 0 else "Critical"
    return condition, score

def irrigation_plan(s):
    if s["rainfall"] >= 12:
        return False, 0.0, "Rainfall is sufficient; irrigation is skipped."
    if s["soil_moisture"] > 55 and s["humidity"] > 65:
        return False, 0.0, "Soil moisture and humidity are adequate."
    stage_factor = {1: 0.8, 2: 1.0, 3: 1.2, 4: 1.1, 5: 0.9}.get(s["crop_stage"], 1.0)
    deficit = max(0, 50 - s["soil_moisture"])
    heat_factor = max(0, s["temperature"] - 25) * 0.7
    humidity_factor = max(0, 65 - s["humidity"]) * 0.25
    litres = (deficit * 1.6 + heat_factor + humidity_factor) * stage_factor
    if s["rainfall"] > 0:
        litres *= 0.6
    litres = min(max(litres, 0), 65)
    return litres > 2, round(litres, 1), "Targeted irrigation recommended."

def to_float(data, key, default, minimum, maximum):
    value = float(data.get(key, default))
    if not minimum <= value <= maximum:
        raise ValueError(f"{key} must be between {minimum} and {maximum}.")
    return value

model = load_model()

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/health")
```

```
def health():
    return jsonify({"status": "ok"})

@app.route("/api/sensors")
def sensors():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()
    return jsonify(pd.read_csv(DATA_FILE).tail(10).to_dict(orient="records"))

@app.route("/api/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json(silent=True) or {}
        s = {
            "soil_moisture": to_float(data, "soil_moisture", 30, 0, 100),
            "temperature": to_float(data, "temperature", 28, -10, 60),
            "humidity": to_float(data, "humidity", 50, 0, 100),
            "rainfall": to_float(data, "rainfall", 0, 0, 500),
            "crop_stage": int(to_float(data, "crop_stage", 3, 1, 5))
        }
        frame = pd.DataFrame([s], columns=FEATURES)
        prediction = bool(model.predict(frame)[0])
        probabilities = model.predict_proba(frame)[0]
        confidence = float(max(probabilities))
        plan_needed, litres, reason = irrigation_plan(s)
        final_need = prediction and plan_needed
        if not final_need:
            litres = 0.0
            if s["rainfall"] >= 12:
                reason = "Rainfall is sufficient; irrigation is skipped."
            elif s["soil_moisture"] > 55 and s["humidity"] > 65:
                reason = "Soil moisture and humidity are adequate."
            else:
                reason = "The model and water-saving rules do not recommend irrigation."
        condition, score = crop_condition(s)
        return jsonify({
            "needs_irrigation": final_need,
            "recommended_liters": litres,
            "confidence": round(confidence, 2),
            "crop_condition": condition,
            "crop_score": score,
            "reason": reason,
            "inputs": s
        })
    except (TypeError, ValueError) as error:
        return jsonify({"error": str(error)}), 400

if __name__ == "__main__":
    app.run(debug=True, host="127.0.0.1", port=5000)
```

Appendix B — templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AI Smart Agriculture</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
  <script src="{{ url_for('static', filename='script.js') }}" defer></script>
</head>
<body>
  <header class="hero">
    <div class="container">
      <p class="eyebrow">SMART FARMING DASHBOARD</p>
      <h1>AI Smart Agriculture</h1>
      <p>Monitor field conditions and explore data-driven irrigation recommendations.</p>
    </div>
  </header>

  <main class="container">
    <section class="panel">
      <div class="section-heading">
        <div><p class="eyebrow">FIELD INPUTS</p><h2>Analyze field conditions</h2></div>
        <span class="badge">Demonstration</span>
      </div>
      <form id="prediction-form">
        <div class="form-grid">
          <label>Soil moisture (%)<input name="soil_moisture" type="number" min="0" max="100"
    value="30" required></label>
          <label>Temperature (°C)<input name="temperature" type="number" min="-10" max="60"
    value="28" required></label>
          <label>Humidity (%)<input name="humidity" type="number" min="0" max="100" value="50"
    required></label>
          <label>Rainfall (mm)<input name="rainfall" type="number" min="0" max="500" value="0"
    required></label>
          <label>Crop growth stage (1–5)<input name="crop_stage" type="number" min="1" max="5"
    value="3" required></label>
        </div>
        <button type="submit" id="analyze-button">Analyze conditions</button>
      </form>
      <p class="hint">Enter sample values or readings from calibrated field sensors.</p>
      <p id="status-message" role="status" aria-live="polite"></p>
    </section>

    <section class="results-grid" aria-live="polite">
      <article class="result-card"><p class="card-label">Irrigation status</p><h3 id="irrigation-
    status">Not analyzed</h3><p id="irrigation-reason">Submit field values to see a
    recommendation.</p></article>
      <article class="result-card"><p class="card-label">Crop condition</p><h3 id="crop-
    condition">—</h3><p>Rule-based demonstration classification</p></article>
      <article class="result-card"><p class="card-label">Recommended water</p><h3><span id="water-
    litres">—</span> <small>L</small></h3><p>Illustrative estimate; validate for your crop and
    field.</p></article>
      <article class="result-card"><p class="card-label">Model confidence</p><h3
    id="confidence">—</h3><p>Confidence of the model's predicted class</p></article>
    </section>

    <section class="panel">
      <div class="section-heading"><div><p class="eyebrow">RECENT READINGS</p><h2>Sensor data
    preview</h2></div><button class="secondary" id="refresh-button"
    type="button">Refresh</button></div>
      <div class="table-wrap"><table><thead><tr><th>Soil
    moisture</th><th>Temp.</th><th>Humidity</th><th>Rainfall</th><th>Stage</th></tr></thead><tbody
    id="sensor-table"><tr><td colspan="5">Loading readings…</td></tr></tbody></table></div>
      <p class="hint">The preview refreshes every 30 seconds. Data is generated for demonstration
    unless you replace the CSV with validated readings.</p>
    </section>

    <section class="notice">
      <h2>Project note</h2>
      <p>This prototype demonstrates a machine-learning workflow. It is not a certified agricultural
     advisory system. Do not use the illustrative water estimate for real irrigation without field
    validation.</p>
    </section>
  </main>
  <footer><div class="container">AI Smart Agriculture Project · Flask + Random Forest</div></footer>
</body>
</html>
```

Appendix C — static/style.css

```
* { box-sizing: border-box; }
:root { font-family: Arial, Helvetica, sans-serif; color: #17352a; background: #f2f7f2; }
body { margin: 0; line-height: 1.5; }
.container { width: min(1100px, calc(100% - 32px)); margin: 0 auto; }
.hero { padding: 42px 0 48px; color: white; background: linear-gradient(120deg, #124b35, #2d7b4e); }
.hero h1 { margin: 8px 0; font-size: clamp(2rem, 5vw, 3.3rem); }
.hero p { max-width: 650px; }
.eyebrow { margin: 0; font-size: .75rem; font-weight: 700; letter-spacing: .13em; text-transform:
    uppercase; opacity: .8; }
main { padding: 28px 0 44px; }
.panel, .result-card, .notice { background: white; border: 1px solid #dce8dc; border-radius: 16px;
    box-shadow: 0 5px 18px rgba(20, 60, 35, .05); }
.panel { padding: 24px; margin-bottom: 22px; }
.section-heading { display: flex; align-items: center; justify-content: space-between; gap: 16px;
    margin-bottom: 20px; }
h2 { margin: 3px 0 0; font-size: 1.4rem; }
.badge { background: #e5f4e8; color: #21623d; padding: 7px 11px; border-radius: 999px; font-size:
    .8rem; font-weight: 700; }
.form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 16px;
    }
label { display: grid; gap: 7px; font-size: .9rem; font-weight: 700; }
input { width: 100%; border: 1px solid #bfd2c2; border-radius: 9px; padding: 11px 12px; font:
    inherit; color: inherit; background: #fcfefc; }
input:focus { outline: 3px solid #c7ebd0; border-color: #2d7b4e; }
button { margin-top: 20px; border: 0; border-radius: 9px; padding: 12px 18px; background: #216a43;
    color: white; font: inherit; font-weight: 700; cursor: pointer; }
button:disabled { opacity: .65; cursor: wait; }
button.secondary { margin: 0; background: #e5f4e8; color: #21623d; }
.hint { color: #60776a; font-size: .85rem; }
#status-message { min-height: 1.2em; color: #9b3b20; }
.results-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap:
    16px; margin-bottom: 22px; }
.result-card { padding: 20px; }
.card-label { margin: 0; color: #60776a; font-size: .88rem; }
.result-card h3 { margin: 9px 0; font-size: 1.55rem; overflow-wrap: anywhere; }
.result-card h3 small { font-size: .9rem; }
.result-card p:last-child { margin-bottom: 0; color: #60776a; font-size: .88rem; }
.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; min-width: 480px; }
th, td { padding: 12px; border-bottom: 1px solid #e6eee6; text-align: left; font-size: .9rem; }
th { color: #41644f; background: #f6faf6; }
.notice { padding: 20px 24px; border-left: 5px solid #d5a23b; }
.notice h2 { margin-bottom: 8px; }
.notice p { margin-bottom: 0; }
footer { padding: 20px 0; background: #e6efe7; color: #486452; font-size: .85rem; }
@media (max-width: 600px) { .hero { padding: 32px 0; } .panel { padding: 18px; } .section-heading {
    align-items: flex-start; } }
```

Appendix D — static/script.js

```
const form = document.getElementById("prediction-form");
const analyzeButton = document.getElementById("analyze-button");
const statusMessage = document.getElementById("status-message");

form.addEventListener("submit", async (event) => {
  event.preventDefault();
  analyzeButton.disabled = true;
  statusMessage.textContent = "Analyzing field conditions...";
  const values = Object.fromEntries(new FormData(form).entries());
  for (const key of Object.keys(values)) values[key] = Number(values[key]);

  try {
    const response = await fetch("/api/predict", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(values)
    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || "Prediction failed.");

    document.getElementById("irrigation-status").textContent =
      data.needs_irrigation ? "Irrigation suggested" : "No irrigation suggested";
    document.getElementById("irrigation-reason").textContent = data.reason;
    document.getElementById("crop-condition").textContent = data.crop_condition;
    document.getElementById("water-litres").textContent = data.recommended_liters.toFixed(1);
    document.getElementById("confidence").textContent = `${(data.confidence * 100).toFixed(1)}%`;
    statusMessage.textContent = "Analysis complete.";
  } catch (error) {
    statusMessage.textContent = error.message || "Could not connect to the server. Check that app.py
     is running.";
  } finally {
    analyzeButton.disabled = false;
  }
});

async function loadSensors() {
  const table = document.getElementById("sensor-table");
  try {
    const response = await fetch("/api/sensors");
    if (!response.ok) throw new Error("Unable to load sensor data.");
    const rows = await response.json();
    table.replaceChildren();
    rows.forEach((row) => {
      const tr = document.createElement("tr");
      [row.soil_moisture, row.temperature, row.humidity, row.rainfall, row.crop_stage]
        .forEach((value) => {
          const td = document.createElement("td");
          td.textContent = value;
          tr.appendChild(td);
        });
      table.appendChild(tr);
    });
  } catch {
    table.innerHTML = '<tr><td colspan="5">Sensor data is unavailable. Check the Flask
    server.</td></tr>';
  }
}

document.getElementById("refresh-button").addEventListener("click", loadSensors);
loadSensors();
setInterval(loadSensors, 30000);
```

Appendix E — requirements.txt

```
Flask
pandas
scikit-learn
joblib
```

Appendix F — README.md

````
# AI Smart Agriculture Project

## Overview
A Flask web application that demonstrates smart irrigation recommendations using a Random Forest
    classifier and sample field readings.

## Features
- Soil moisture, temperature, humidity, rainfall, and crop-stage inputs
- Machine-learning irrigation prediction and model confidence
- Rainfall-aware water-saving rules
- Rule-based crop-condition category
- Recent sensor-data table with automatic refresh
- JSON endpoints for health, sensor data, and prediction

## Project structure
```text
AI_Smart_Agriculture/
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ templates/
■   ■■■ index.html
■■■ static/
    ■■■ style.css
    ■■■ script.js
```

## Run locally
1. Install Python 3.10 or newer.
2. Open a terminal in this project folder.
3. Install dependencies: `python -m pip install -r requirements.txt`
4. Start the application: `python app.py`
5. Open `http://127.0.0.1:5000` in a browser.

The app generates `sensor_data.csv` and `irrigation_model.pkl` on first run.

## Important limitation
This is a learning prototype using generated demonstration data and heuristic rules. The displayed
    confidence and water quantity are not validated agronomic recommendations. Replace sample data
    with calibrated sensor readings and validated crop-specific data, then evaluate with
    agricultural experts before real-world use.
````

AI SMART AGRICULTURE

Additional Project Explanation

This section provides a little more detail about the purpose, working process, code modules, expected benefits, testing approach and future development of the AI Smart Agriculture project. It supplements the original project document and keeps the same prototype-based scope.

1. Detailed Project Overview

AI Smart Agriculture is a browser-based learning project that demonstrates how field measurements can be used to support irrigation planning. The user enters soil moisture, temperature, humidity, rainfall and crop growth stage in the dashboard. The backend processes these values and combines a machine-learning prediction with simple water-saving rules. The result is shown on the webpage in a clear format so that a user can understand whether irrigation is suggested and why.

2. Why This Project Is Useful

Farmers need to consider changing field and weather conditions before watering crops. If irrigation is applied when the soil already has enough moisture or rainfall is sufficient, water may be wasted. If the soil is too dry, crop growth may be affected. This prototype demonstrates how several readings can be considered together instead of making a decision from only one measurement. Its main educational purpose is to show how data, software and AI techniques can be combined in a practical problem.

3. Explanation of the Input Parameters

Soil moisture: Indicates how much water is present in the soil. Low moisture may indicate that the field needs attention.
Temperature: Represents the surrounding temperature and helps the prototype account for hot conditions.
Humidity: Represents moisture in the air and is used alongside other measurements.
Rainfall: Helps the water-saving rule avoid suggesting irrigation when the entered rainfall is at or above the prototype threshold.
Crop growth stage: A value from 1 to 5 that changes the illustrative water estimate through a stage factor.

4. How the AI Model Works

The Python backend creates a demonstration dataset when no sensor CSV file is present. It uses the input measurements as features and the generated irrigation label as the target. A Random Forest Classifier is trained using this data. Random Forest combines predictions from multiple decision trees to classify the input. The application then obtains the predicted class and its highest class probability. The probability is displayed as model confidence; it should not be interpreted as proof that the prediction is correct in a real farm.

5. Role of the Water-Saving Rules

The machine-learning output is not used alone. A separate irrigation-plan function checks rainfall and whether soil moisture and humidity are already high. If the rules indicate that watering should be skipped, the displayed water quantity is set to zero. Otherwise, the prototype calculates an illustrative quantity using soil-moisture deficit, temperature, humidity, crop stage and rainfall. This rule layer makes the program easier to explain, but its thresholds and formula are examples only and require agricultural validation before any real use.

6. Explanation of the Crop-Condition Result

The crop-condition function assigns a score using soil moisture, temperature, humidity and rainfall. Based on the total score, the dashboard displays one of three categories: Healthy, Moderate or Critical. This is a transparent rule-based classification, separate from the Random Forest irrigation model. It helps demonstrate how a program

can convert several readings into a simple status label. It does not diagnose plant disease or replace inspection by a farmer or agricultural specialist.

7. Explanation of Each Code File

app.py: The main Python Flask file. It creates sample records, trains or loads the model, validates inputs, calculates results and provides API routes.
templates/index.html: Builds the webpage structure, input form, result cards and recent sensor table.
static/style.css: Controls the colours, spacing, cards, form layout and responsive design for smaller screens.
static/script.js: Reads form values, sends them to the Flask API, displays the returned results and refreshes the sensor preview periodically.
requirements.txt: Lists the Python packages needed to run the project.
README.md: Explains the project structure and setup steps.

8. Data Flow and Output

First, the user enters the five field values in the webpage. JavaScript sends these values to the POST /api/predict endpoint as JSON. Flask checks that the values fall within the accepted ranges, passes the measurements to the model and applies the irrigation rules. The backend returns the result as JSON. JavaScript then updates the irrigation status, reason, crop-condition category, estimated water quantity and model confidence without reloading the page. The GET /api/sensors endpoint supplies recent sample rows for the table, while GET /api/health checks whether the backend is responding.

9. Suggested Testing

The prototype can be tested with different sample inputs. For example, enter low soil moisture, high temperature, low humidity and little rainfall, then observe the recommendation. Next, enter rainfall at or above the configured threshold and check that the water-saving rule skips irrigation. Also try values outside the accepted ranges to confirm that the application returns a validation message. These tests check the software workflow only; they do not prove that the recommendations are suitable for a real crop.

10. Benefits and Learning Outcomes

This project demonstrates a complete beginner-friendly AI web application. It provides practice with Python programming, Flask routing, tabular data handling with Pandas, machine learning with Scikit-learn, HTML page structure, CSS styling, JavaScript fetch requests and JSON communication. It also illustrates input validation, modular code design and the importance of explaining model limitations. The dashboard makes the output easier to understand than a command-line-only program.

11. Limitations and Future Scope

The current application uses generated sample records rather than readings from physical sensors. The labels are created by simple rules, so the reported test accuracy measures performance against those generated labels and cannot establish real agricultural accuracy. The water quantity is only an illustrative estimate. Future work could connect calibrated soil-moisture sensors and a weather service, collect verified data for a specific crop and soil type, evaluate the model on independent field data, add historical charts and alerts, improve API security, and deploy the Flask backend to a suitable Python hosting platform. Agricultural specialists should review any real-world irrigation advice.

12. Conclusion

AI Smart Agriculture shows how environmental measurements, a Random Forest classifier and transparent water-saving rules can work together in a web dashboard. The project is suitable as an educational prototype for demonstrating an AI-assisted irrigation workflow. With validated field data, calibrated sensors, careful testing and expert review, it could be developed further. Until then, its predictions and water estimates should be treated as demonstrations rather than actual farming instructions.

Important note: This is a learning prototype. Do not use its sample predictions or illustrative water estimates to make real irrigation decisions without independent validation.